Lab Session 1: Password Auditing & Hash Cracking — Part A: Offline Hash Cracking with John the Ripper

Date: 8 June 2026

Duration: 2 hours

Scenario:

During a simulated forensic investigation, I recovered /etc/passwd and /etc/shadow files from a compromised Linux server. The shadow file contained user password hashes that needed to be audited for weak credentials. Part A covers the offline hash cracking portion of this investigation.

Objective:

Understand how password hashes are stored on Linux systems, how they can be extracted, and how offline cracking tools like John the Ripper can be used to recover plaintext passwords from those hashes. This part focused specifically on the offline cracking methodology.

Environment:

Kali IP: 192.168.56.1/24

Virtual Machines: Kali Linux

Subnet: 192.168.56.0/24

Tools Used: John The Ripper

Target: Practice Shadow File (test user) + Kali System hashes

Wordlists: rockyou.txt- /usr/share/wordlist/rockyou.txt

Hash Type: SHA-512 (6) – Linux Shadow file format

Commands, step by step:

1. MYHASH=$(openssl passwd -6 password123)
   echo "testuser:$MYHASH:18000:0:99999:7:::" > test_shadow.txt
2. Echo 'testuser:x:1001:1001::/home/testuser:/bin/bash' > test_passwd.txt
   unshadow test_passwd.txt test_shadow.txt > crackme.txt
   cat crackme.txt
3. john crackme.txt
4. sudo gzip -d /usr/share/wordlists/rockyou.txt.gz
   john --wordlist=/usr/share/wordlists/rockyou.txt crackme.txt
5. john --show crackme.txt
6. sudo unshadow /etc/passwd /etc/shadow > combined_hashes.txt

Background Info

Tools like John the Ripper do not reverse the hash , they guess. The tool takes a wordlist, hashes each candidate using the same algorithm, and compares the result to the target hash. When they match, the original password has been found. This is a dictionary attack. The speed of cracking depends heavily on the hashing algorithm. Modern Linux systems use SHA-512 with 5,000 iterations — deliberately slow to make cracking computationally expensive.

1.3 The /etc/shadow Format

On Linux, /etc/shadow stores one entry per user:

username:6salt$ hashedpassword:lastchange:min:max:warn:inactive:expire:

- 6 — SHA-512 algorithm was used
- salt — random value added before hashing (prevents identical passwords from producing identical hashes)

- hashedpassword — the actual resulting hash

John the Ripper needs both /etc/passwd and /etc/shadow combined into a single file. The unshadow utility handles this.

Building the Practice Hash Environment

Rather than working directly on real system files (which risks locking myself out), I created a safe practice environment with a fake user and a real hash.

Findings:

- ***MYHASH=$(openssl passwd -6 password123)*** This successfully generated a secure SHA-512 password hash without relying on external programming libraries. Storing the output directly into a shell variable solved manual data transcription errors while simulating the exact structural formatting required for system authentication records. This step verified that utilizing a strong algorithm prevents standard credential mapping.
- ***Echo 'testuser:x:1001:1001::/home/testuser:/bin/bash' > test_passwd.txt***
  unshadow test_passwd.txt test_shadow.txt > crackme.txt
  cat crackme.txt I found out that John the Ripper's unshadow utility requires a corresponding passwd-format file. I created a minimal one and combined both files. The combined entry appeared correctly - testuser with the full SHA-512 hash and directory path..
- ***john crackme.txt*** John loaded the hash successfully (on the very first attempt the placeholder text was still in the file, which was another lesson learned). It detected SHA-512, spun up 4 OpenMP threads, and began cycling through candidate passwords at roughly 1,380–1,430 per second. After approximately one hour it was still running - John had moved from Single mode through the wordlist into incremental (brute force) mode, which is essentially infinite.
- ***sudo gzip -d /usr/share/wordlists/rockyou.txt.gz***
  john --wordlist=/usr/share/wordlists/rockyou.txt crackme.txt After switching to rockyou.txt , ‘password 123’ was found pretty quickly, which indicated a success.

- ***sudo unshadow /etc/passwd /etc/shadow > combined_hashes.txt*** As an exploratory step, unshadow was also run against the actual Kali system files.This produced a real combined hash file from the live Kali installation. The practice file workflow was documented first to demonstrate understanding of the process from scratch rather than pulling existing system files directly.

Error and Troubleshooting

The password auditing workflow encountered several technical and environmental constraints that required resolution. An initial failure to load password hashes occurred because placeholder text was mistakenly left in the target file, which was corrected by generating a valid SHA-512 hash string. Attempts to automate hash creation via script triggered a script compilation failure because Python 3.13 removes the legacy crypt module; this was bypassed by shifting directly to native OpenSSL commands. Operational inefficiencies also arose when a default John the Ripper scan ran fruitlessly for over an hour against the intensive SHA-512 algorithm, prompting a shift to a targeted dictionary attack. Finally, a resource availability issue was resolved by manually decompressing the default Kali wordlist from its archival format, successfully enabling the dictionary attack to proceed.

Lessons Learned

- Always build and test against a disposable practice file before touching real system hashes - it avoids the risk of locking yourself out.
- Know your tools' dependencies before relying on them - Python 3.13 quietly dropped the crypt module, and OpenSSL was a faster, more reliable substitute.
- Default cracking modes are not always the right first move - a targeted wordlist beats brute force against strong, slow hashing algorithms.

- Common wordlists like rockyou.txt remain effective in 2026 because people still reuse the same weak passwords.
- Document errors as they happen - the crypt module removal and the one-hour brute force timeout were as educational as the successful crack.

Note to Self:

Always build the practice file properly before running John - a placeholder hash wastes a whole run. Also: check whether a Python module still exists in the current version before trusting an old lab sheet, and remember that brute force against SHA-512 is a dead end without a wordlist.

What's Next?

Part B — Online Brute Forcing with Hydra (SSH and FTP against Metasploitable 2).